FADENA
FACULTAD DE DEFENSA NACIONAL

SQL Avanzado

Temario

- Sentencias de Manipulación de Datos (DML):
  - Subconsultas (consultas anidadas).
    - Operadores: comparación, IN, NOT IN, ANY, ALL, EXISTS, NOT EXISTS.
  - Operadores de Conjunto: UNION, INTERSECT, EXCEPT.
- Sentencias de Definición de Datos (DDL): Vistas, Índices, otros.
- Sentencias de Control de Datos (DCL).
- Sentencias de Control de Transacciones (TCL).

SUBCONSULTAS (DML)

- Una **subconsulta**, o **consulta anidada**, **es una sentencia SELECT dentro de otra sentencia SQL**.
- Puede utilizarse dentro de cualquier otra sentencia DML, como una **SELECT** o bien en un **INSERT**, **UPDATE** o **DELETE**.
- Las subconsultas pueden anidarse unas dentro de otras tanto como sea necesario.
- Tiene la misma sintaxis que una sentencia SELECT normal, exceptuando que **suele aparecer encerrada entre paréntesis**.
- Podemos llamar **consulta externa o principal** a aquella que incluye a la subconsulta.
- Se usan para simplificar consultas complejas al dividirlas en partes manejables, y realizar operaciones que de otro modo serían difíciles o imposibles.

SUBCONSULTAS (DML)

- Como cualquier SELECT, **pueden retornar**: **un único valor** (escalar), **una única fila**, **múltiples filas**. Cada una de estas variantes se aplicarán según sea el caso.

Las subconsultas **se pueden encontrar en diferentes cláusulas**:

- **En WHERE**: para filtrar registros basándose en los valores de la subconsulta.
- **En HAVING**: para filtrar grupos de datos según un criterio que de pende de la subconsulta.
- **En FROM**: para utilizar el resultado de la subconsulta como una tabla temporal.
- **En SELECT**: para incluir un valor calculado por la subconsulta en el resultado.

Subconsultas en cláusulas WHERE y HAVING (DML)

- El predicado que se evalúa para realizar una restricción en estas cláusulas está formado por **comparaciones** unidas por AND/OR.
- Cada comparación involucra dos **operandos** que son: columnas, constantes (o expresiones que contengan ambas), y ahora veremos también **resultados de subconsultas**.
- **Operadores** que se pueden utilizar con las **subconsultas**:
  - **Operadores básicos de comparación** (=, <>, >, <, >=, <=).
  - **IN, NOT IN.**
  - **ANY, ALL.**
  - **EXISTS y NOT EXISTS.**

Subconsultas en cláusulas WHERE y HAVING (DML)

Operadores Básicos de Comparación (=, <>, >, <, >=, <=)

expresion operador (subconsulta)

La subconsulta debe devolver un solo valor. El predicado **será verdadero si la comparación indicada por el operador entre la expresión y el resultado de la subconsulta es verdadera**.

```
SELECT *
  FROM producto
WHERE precio >= (SELECT MAX(precio)
                 FROM producto P INNER JOIN fabricante F
                   ON P.codigo_fabricante  = F.codigo
                 WHERE fabricante.nombre = 'Samsung');
```

```
SELECT departamentoID, AVG(sueldo)
  FROM empleados
GROUP BY departamentoID
HAVING AVG(sueldo) > ( SELECT AVG(sueldo)
                       FROM empleados );
```

Subconsultas en cláusulas WHERE y HAVING (DML)

Operadores IN y NOT IN

expresion [NOT] IN (subconsulta)

La subconsulta puede devolver varias filas.

En el caso de **IN**, el predicado **será verdadero si la expresión es igual a alguno de los valores de la columna devuelta por la subconsulta**.

En el caso de **NOT IN**, el predicado **será verdadero si la expresión es distinta a todos los valores de la columna devuelta por la subconsulta,** o si esta no devuelve ninguna fila.

Ejemplos:

```
SELECT ciudadID, nombre
  FROM ciudad
WHERE ciudadID IN ( SELECT ciudadID FROM clientes);

SELECT *
  FROM cliente
WHERE clienteID NOT IN (SELECT clienteID FROM pedido);
```

Subconsultas en cláusulas WHERE y HAVING (DML)

Operadores ALL y ANY : Se utilizan con operadores de comparación (=, <>, >, <, >=, <=)

expresion operador [ANY | ALL] (subconsulta)

La subconsulta puede devolver varias filas.

En el caso de **ANY**, el predicado **será verdadero** si la comparación es verdadera **para alguno** de los valores de la columna devuelta por la subconsulta.

En el caso de **ALL**, el predicado **será verdadero si la comparación es verdadera para todos los valores de la columna devuelta por la subconsulta,** o si esta no devuelve ninguna fila.

Ejemplo:

```
SELECT empleadoID, nombre
  FROM empleados
WHERE categoria = 'operario'
    AND sueldo > ANY (SELECT sueldo
                      FROM empleados
                      WHERE categoria = 'supervisor');
```

(*) En este caso recuperará los "operarios" cuyo sueldo sea mayor al de **algún** "supervisor". Si usáramos **ALL**, recuperará los "operarios" con sueldo mayor a **todos** los "supervisores".

Subconsultas en cláusulas WHERE y HAVING (DML)

Sobre los operadores de subconsultas vistos (operadores de comparación, IN, NOT IN, ANY, ALL)

Se pueden usar también comparando más de un valor:

(expr1, expr2, …) operador (subconsulta)

(expr1, expr2, …) [NOT] IN (subconsulta)

(expr1, expr2, …) operador [ANY | ALL] (subconsulta)

En estos casos, **la subconsulta debe devolver tantas columnas como las especificadas entre paréntesis a la izquierda**. Las comparaciones, se harán por cada campo en las filas de ambos lados.

Ejemplo:

```
SELECT *
  FROM facturas
WHERE (descuento, monto) = ( SELECT MAX(descuento),
                                    MAX(monto)
                             FROM facturas );
```

Subconsultas en cláusulas WHERE y HAVING (DML)

Sobre los operadores de subconsultas vistos (NOT IN y ALL)

En estos casos en particular, **si la subconsulta devuelve algún nulo**, el predicado se evaluará como nulo (más allá de que existan otras filas con valor), y **la consulta externa no devolverá registros**.

La solución es incluir en la subconsulta una condición de **"IS NOT NULL"** para los campos a devolver, o usar funciones como **COALESCE**(columna, valor_si_es_nulo); que permite devolver un valor especificado para las filas donde columna sea nulo.

Ejemplo:

```
SELECT *
FROM facturas
WHERE descuento >= ALL ( SELECT COALESCE(descuento, 0)
                         FROM facturas);
```

Subconsultas en cláusulas WHERE y HAVING (DML)

Operadores EXISTS y NOT EXISTS

EXISTS (subconsulta)

NOT EXISTS (subconsulta)

La subconsulta hace referencia a columnas o expresiones de la consulta externa para su ejecución.

En el caso de **EXISTS**, el predicado **será verdadero si la subconsulta devuelve al menos una fila**.

En el caso de **NOT EXISTS**, el predicado **será verdadero si la subconsulta no devuelve ninguna fila**.

Ejemplos:

```
SELECT clienteID, nombre FROM cliente C
WHERE EXISTS ( SELECT pedidoID
               FROM pedido P
               WHERE C.clienteID = P.clienteID
               AND P.fecha > '2025-08-10');

SELECT * FROM cliente C
WHERE NOT EXISTS (SELECT id_pedido
                  FROM pedido P
                  WHERE C.clienteID = P. clienteID);
```

Subconsultas Correlacionadas y No Correlacionadas

Subconsulta no correlacionada:

- **Se ejecuta independientemente de la consulta externa**, recuperando datos sin depender de referencias a sus datos.
- La **subconsulta se ejecuta primero (una sola vez)** y luego la consulta externa. Esto significa que el resultado de la subconsulta se utiliza como entrada para la consulta externa.

Subconsulta correlacionada:

- **Depende de los resultados de la consulta externa**, ya que **hace referencia a columnas o valores de esta**.
- **La consulta externa se ejecuta primero** y, a continuación, **la consulta interna para cada fila devuelta por esta**.

Ejemplo de subconsulta no correlacionada:

```
SELECT ciudadID, nombre
  FROM ciudad
WHERE ciudadID IN ( SELECT ciudadID FROM clientes);
```

Ejemplo de subconsulta correlacionada:

```
SELECT e1.nombre, e1.sueldo
      FROM empleados e1
   WHERE sueldo > ( SELECT AVG(salario)
                    FROM empleados e2
                   WHERE e2.deptoID = e1.deptoID );
```

Subconsultas en la cláusula FROM (DML)

- Es posible incluir subconsultas en la cláusula **FROM**, **para realizar una consulta sobre la tabla que se obtiene como resultado de ejecutar otra consulta.**
- Son útiles para simplificar consultas de múltiples pasos.
- La subconsulta **resulta en una tabla (temporal o derivada). Se le debe dar un alias** mediante la cláusula AS, para poder referenciarla.
- **La subconsulta puede devolver tantas filas y columnas como sea necesario**.

Ejemplo:

```
SELECT MAX(promedio.precio_promedio)
  FROM ( SELECT categoria,
                AVG(precio) AS precio_promedio
           FROM producto
          GROUP BY categoria) AS promedio;
```

Subconsultas en la cláusula SELECT (DML)

- Las subconsultas pueden aparecer en la cláusula SELECT. Son útiles para mostrar información agregada relacionada a un registro devuelto.
- En este caso, **tienen que ser subconsultas de tipo escalar, que devuelven una única fila y columna (un solo valor)**.

Ejemplo:

```
SELECT nombre,
        ( SELECT AVG(salario)
            FROM empleados AS e2
           WHERE e2.departamento = e1.departamento ) AS salario_prom_dep
  FROM empleados e1;
```

Subconsultas en sentencias INSERT (DML)

- **El resultado de una subconsulta puede usarse como entrada de una sentencia INSERT**.
- Las columnas devueltas por la subconsulta deben coincidir con las con las especificadas en la sentencia INSERT INTO.
- Suele ser útil para crear copias de tablas, o para rellenar tablas con datos de otras basándose en criterios específicos.

Ejemplo:

```
INSERT INTO aprobadosMatem (alumnoID, nombre, nota)
    SELECT a.alumnoID, a.nombre, c.nota
      FROM alumnos a INNER JOIN calificaciones c ON (a.alumnoID = c.alumnoID)
    WHERE c.nota > 6 AND c.materia = 'Matem';
```

(*) En el uso con INSERT, la subconsulta puede no estar encerrada entre paréntesis.

Subconsultas en sentencias UDATE y DELETE (DML)

- Como las sentencias **UPDATE y DELETE** pueden usarse con la cláusula WHERE, estas **pueden utilizar las subconsultas** tal cual se mostraron en las sentencias SELECT.

Ejemplo UPDATE:

```
UPDATE empleados
    SET sueldo = sueldo * 1.1
WHERE departamentoID IN ( SELECT departamentoID
                          FROM departamentos
                          WHERE presupuesto > 80000);
```

Ejemplo DELETE:

```
DELETE FROM empleados
WHERE sueldo < (SELECT AVG(sueldo) FROM empleados);
```

OPERADORES DE CONJUNTO (DML)

- La **UNION**, la **INTERSECCIÓN** y la **DIFERENCIA** son operadores de álgebra relacional que se pueden usar en SQL. La sintaxis para eso es:

```
sentencia_SELECT
UNION | INTERSECT | EXCEPT [ ALL ]
sentencia_SELECT
[ ORDER BY columna [ ASC | DESC ] [,columna [ ASC | DESC ] ];
```

- **Las sentencias SELECT involucradas deben devolver el mismo número de columnas**, **y** las columnas correspondientes en ambas sentencias deberán ser **del mismo tipo de datos**.
- La cláusula ORDER BY sólo puede aparecer una vez en la consulta, al final de esta. La ordenación se realizará sobre el resultado de la unión, intersección o diferencia.

OPERADORES DE CONJUNTO (DML)

Operador UNION : Devuelve todas las filas que devuelve la **primera sentencia SELECT, más aquellas filas de la segunda** sentencia SELECT que no han sido ya devueltas por la primera.

```
SELECT ciudad FROM clientes
UNION
SELECT ciudad FROM vendedores;
```

Operador INTERSECT : Devuelve las filas que se encuentran tanto en el resultado de la **primera sentencia SELECT como en el de la segunda** sentencia SELECT.

```
SELECT ciudad FROM clientes
INTERSECT
SELECT ciudad FROM vendedores;
```

Operador EXCEPT : Este operador (que representa la diferencia o minus) devuelve las filas que se encuentran en el resultado de la **primera sentencia SELECT y no se encuentran en el resultado de la segunda** sentencia SELECT.

```
SELECT ciudad FROM clientes
EXCEPT
SELECT ciudad FROM vendedores;
```

(*) Para los tres casos, **en el resultado no se muestran duplicados**. Se puede **evitar la eliminación de duplicados** especificando la **palabra clave ALL**.

VISTAS (DDL)

- Una vista es una **tabla virtual basada en el resultado de una consulta SQL**.
  Su contenido está definido como una consulta sobre una o varias tablas:

```
CREATE VIEW nombre_vista [ ( nombre_col, ... ) ]
        AS sentencia_SELECT;
```

- Las columnas de la vista se pueden nombrar especificando la lista entre paréntesis.
  Si no, toma los mismos que los de las columnas de la sentencia SELECT.
- **No almacena datos físicamente**, los datos existen en una o más tablas referenciadas en la consulta. Las vistas se actualizan dinámicamente, si los datos de las tablas subyacentes cambian, las vistas que se basan en esas tablas también lo harán.

VISTAS (DDL)

- **Seguridad:** permiten ocultar partes de la base de datos a ciertos usuarios. Por ejemplo, atributos (columnas) que se han omitido al definir una vista.
- **Reducción de complejidad y reutilización:** permiten simplificar consultas complejas; y guardarlas como vistas referenciándolas luego para su uso repetitivo, simplificando el código.
- **Independencia de datos a nivel lógico:** Si se añade un atributo a una tabla, o si una tabla existente se reorganiza o se divide en varias tablas, se pueden crear vistas para que los usuarios la sigan viendo igual.

Ejemplo creación:

```
CREATE VIEW v_emp_jef ( empID, empNombre, jefID, jefNombre )
    AS SELECT e.empID, e.nombre, j.empID, j.nombre
        FROM empleados e
         LEFT OUTER JOIN empleados j ON (e.jefeID = j.empID);
```

Ejemplo uso:

```
SELECT v.ventasID, v.fecha, vej.empNombre, vej.jefNombre
  FROM ventas v
 INNER JOIN v_emp_jef vej ON (v.empleadoID = vej.empID)
WHERE ...;
```

ÍNDICES (DDL)

- Los índices son estructuras que se utilizan **para acelerar las operaciones de recuperación de datos de las tablas** con ciertas condiciones de búsqueda**, basados en uno o más campos**.
- En lugar de escanear toda la tabla fila a fila, se puede utilizar los índices para encontrar las filas necesarias mucho más rápidamente (similar al índice de un libro).

```
CREATE INDEX nombre_índice ON tabla (columna1, columna2, ...);
```

- **(*) Crear índices excesivos puede tener el efecto contrario,** ralentizando el rendimiento de la base de datos. Cada índice requiere **espacio de almacenamiento** y **aumenta el tiempo necesario para insertar, actualizar o eliminar datos** (para el mantenimiento de los índices).

ÍNDICES – Clasificación (DDL)

Índices agrupados (clustered): Determinan el orden físico de los datos de una tabla.

- **Las filas se almacenan en el disco en el orden** de la columna o columnas indexadas.
- Los datos pueden ordenarse físicamente de **una** manera -> **sólo puede haber un índice agrupado por tabla**.
- Generalmente se crean para la clave primaria de la tabla. Usa la palabra reservada CLUSTERED.

Índices no agrupados (non clustered): No afectan al orden físico de los datos.

- Crean una **estructura independiente que contiene punteros** a las filas de datos reales.
- **Una tabla puede tener varios índices no agrupados**, cada uno utilizado en un patrón de consulta distinto.

Índices únicos: Garantiza que todos los valores de la columna o las columnas indexadas sean únicos.

Refuerza la integridad de los datos impidiendo valores duplicados. Usa la palabra reservada UNIQUE.

Índices compuestos: Un índice compuesto es aquel que abarca más de una columna.

ÍNDICES (DDL)

Para **encontrar un equilibrio** entre las ventajas de la indexación y sus posibles inconvenientes:

- Indexar las columnas que se utilizan con frecuencia (por ejemplo, en cláusulas WHERE, JOIN y ORDER BY).
- Centrarse en las columnas donde los valores suelen ser distintos (alta cardinalidad). Si una columna tiene pocos valores posibles, un índice sobre esta no aportará ventajas de rendimiento significativas.
- No crear índices para tablas pequeñas (con pocos registros).

Ejemplos de creación de índices:

```
CREATE UNIQUE INDEX idx_correo ON empleados (email);

CREATE INDEX idx_nombrecompleto ON empleados (apellido, nombre);
```

Otros objetos de Bases de Datos (DDL)

Bloques de código SQL pre compilados definidos por el usuario, que se almacenan en la base de datos y se ejecutan bajo demanda. Permiten reutilizar ese código en diferentes momentos.

Procedimientos almacenados: Pueden devolver valores (o no), aceptar parámetros de entrada y salida, y realizar operaciones de modificación de datos (INSERT, UPDATE, DELETE). Generalmente se usan para ejecutar lógica de negocio. No pueden usarse directamente en sentencias SQL directamente, son ejecutadas mediante comandos CALL o EXEC (EXECUTE).

Funciones: Devuelven un valor y solo pueden tener parámetros de entrada. Se utilizan principalmente en sentencias SQL como parte de una expresión, por ejemplo, en la cláusula SELECT o WHERE. No puede actualizar datos, pero puede ser embebida directamente en consultas SQL.

SENTENCIAS DE CONTROL DE DATOS (DCL)

Data Control Language (DCL) es la parte de SQL dedicada a la seguridad, utilizado para controlar los permisos y el acceso a la base de datos. Incluye dos comandos principales: **GRANT** y **REVOKE.**

GRANT: **Se utiliza para proporcionar privilegios**. Los privilegios determinan las **acciones** que un **usuario (o rol)** puede realizar en la base de datos.

- Las acciones (privilegios) pueden ser: **SELECT, INSERT, UPDATE, DELETE, EXECUTE**, etc.
- Los objetos a los que se otorgan los privilegios pueden ser: **tablas**, **vistas**, **procedimientos almacenados**, etc.

GRANT <privilegios> **ON** <objeto> **TO** <usuario>; **Ejemplo:** **GRANT** SELECT, INSERT, DELETE **ON** empleados **TO** usuarioA;

REVOKE: Se utiliza para quitar los privilegios otorgados a un usuario. La sintaxis es similar a la anterior:

REVOKE <privilegios> **ON** <objeto> **FROM** <usuario>;

SENTENCIAS DE CONTROL DE TRANSACCIONES (TCL)

Transaction Control Lenguage (**TCL**) se utiliza para **administrar las transacciones** en la base de datos.

Una transacción SQL es una secuencia de una o más sentencias (INSERT, UPDATE o DELETE) que se consideran una **única unidad de trabajo única e indivisible** (no tiene sentido que se ejecute una sin que se ejecuten las demás).

Los principios **ACID** garantizan que sean procesadas de manera confiable:

- **A (Atomicidad):** Una transacción es una unidad de trabajo indivisible. O bien se completan todos sus pasos, o bien ninguno de ellos.
- **C (Consistencia):** Una transacción debe llevar la base de datos de un estado válido a otro estado válido.
- **I (Aislamiento):** Las transacciones concurrentes no deben interferir entre sí.
- **D (Durabilidad):** Una vez que una transacción ha sido confirmada, sus cambios son permanentes y no se perderán, incluso en caso de un fallo del sistema.

SENTENCIAS DE CONTROL DE TRANSACCIONES (TCL)

Sentencias principales del TCL:

- **BEGIN:** Marca el inicio de una transacción. Todas las operaciones posteriores formarán parte de esta transacción.
- **COMMIT:** Guarda de forma permanente todos los cambios realizados en la transacción. Una vez que se ejecutado, los cambios no se pueden deshacer con ROLLBACK.
- **ROLLBACK:** Deshace todas las operaciones desde el último COMMIT o SAVEPOINT. Es útil para revertir la base de datos a un estado anterior en el caso de un error.
- **SAVEPOINT:** Es una forma de tener un ROLLBACK a un punto específico en la transacción en lugar de deshacer toda la transacción. Es útil en situaciones donde se tiene una transacción muy larga.

Ejemplo básico:

```
BEGIN TRANSACTION; -- Inicia la transacción
-- Lleva a cabo operaciones (en este caso actualiza 2 registros de cuentas)
UPDATE cuentas SET balance = balance - 100 WHERE cuentaID = 1;
UPDATE cuentas SET balance = balance + 100 WHERE cuentaID = 2;
COMMIT; -- Finaliza la transacción
```